

AI/ML CHALLENGES

Challenge 1: AI-Driven Automated Interviewer for Project Presentations

Objective

Build an AI system that listens to a student presenting a project (screen share + speech) and conducts an adaptive interview based on content and responses.

Functional Requirements

- **Presentation Understanding**
 - Extract content from screens using OCR.
 - Transcribe student speech using STT.
 - Analyze UI, code snippets, slides, diagrams.
- **Dynamic Interviewing**
 - Generate context-aware questions from extracted content.
 - Ask follow-up questions based on responses and screen content.
- **Evaluation & Feedback**
 - Score student on:
 - Technical depth
 - Clarity of explanation
 - Originality
 - Understanding of implementation

Deliverable

A live demo where a student presents a project, the system interviews them in real time, and produces a score + feedback report.

Challenge 2: Ultra-Lightweight Client-Side AI for Voice Interview Summaries

Objective

Develop a <30MB client-side AI model that generates filler phrases and real-time summaries to enhance automated voice interviews in the browser.

Requirements

- **Constraints**
 - Model size $\leq 30\text{MB}$
 - Fully client-side (ONNX / WASM)

- Inference latency <50ms
- **Tasks**
 - Real-time summarization of conversation text
 - Detect pauses in text-to-speech responses
 - Insert contextual filler phrases using same TTS output
- **Success Criteria**
 - Runs offline
 - No perceivable lag
 - Accurate, meaningful filler + summary generation

Deliverable

A working web demo displaying live summary updates, filler playback, load time, and response time metrics.

Challenge 3: Learning Recommendation System

Objective

Build an end-to-end recommendation system that suggests next learning items (courses, exercises, resources) based on student interaction data.

Requirements

- **Data & Problem Setup**
 - Choose dataset (listed in Dataset section)
 - EDA: sparsity, distributions, cold-start analysis
 - Temporal train/val/test split
 - Feature engineering on user, item, and temporal context
- **Modeling**
 - Required baselines:
 - Most Popular
 - Collaborative Filtering or Session-based model
 - Advanced Models (choose ≥ 1):
 - Sequence models (RNN/Transformer/SASRec)
 - Hybrid models
 - Candidate generation + ranking pipeline
- **Evaluation**
 - Ranking metrics (Precision@K, Recall@K, NDCG@K, MRR)
 - Coverage, novelty, cold-start analysis
 - Error analysis with qualitative examples
- **Application (Choose One)**
 - Web app (Streamlit/Flask/FastAPI)
 - CLI tool with batch inference

- **Reproducibility**
 - Public repo
 - Training scripts + preprocessing
 - README + Dockerfile recommended

Deliverables

- Codebase with preprocessing, training, evaluation, inference
 - Model artifacts or reproducible scripts
 - Demo app or CLI
 - Report summarizing approach, metrics, and limitations
-

FULLSTACK DEVELOPER CHALLENGES

Challenge 4: Progressive Student Dashboard (Full-Stack)

Objective

Build a web application that tracks student progress across courses, recommends next steps, and visualizes learning insights.

Core Features

- Email authentication, student + mentor roles
- Dashboard showing:
 - Completed lessons
 - Time spent
 - Progress per course
- Visualizations:
 - Trend chart (time series)
 - Pie/donut (distribution or completion status)
- Backend API for:
 - Auth
 - Aggregates & time-series data
 - Lesson details
 - Activity events
- Seeded sample data + clear setup instructions

Stretch Features

- Adaptive recommendations

- Export to CSV
- Mentor dashboards
- Tests + responsive UI

Deliverables

- Full-stack repo + seed data
 - API documentation + screenshots
-

Challenge 5: Live Classroom Collaboration Platform

Objective

Create a lightweight real-time collaboration tool for classroom learning with co-solving capabilities, shared notes, polling, chat, and low-latency sync.

Functional Requirements

- Real-time shared content (notes, chat, co-solving)
- Live feedback + polling
- Session persistence & exports
- Secure access control

Constraints

- Low latency
- Lightweight deployment (free tier friendly)
- Privacy-first design

Deliverables

- Working multi-user demo
 - Synced notes + live poll visualization
 - Exportable session summary
-

Challenge 6: Multi-Source Learning Content Ingestion & Structured Output Generation

Objective

Ingest content from PDFs, videos, documents, transcripts, and generate structured learning artifacts (flashcards, summaries, concept graphs).

Functional Requirements

- Parse multiple file types
- Extract key concepts + topic hierarchy
- Auto-generate structured educational outputs
- Store and enable retrieval by topic

Deliverables

- CLI or web tool ingesting files
- JSON/CSV flashcards
- Concept graph or learning path

Challenge 7: Turn Recurring Meeting Knowledge Into Training Modules

Objective

Convert recurring meeting notes into reusable training modules (tutorials, FAQs, how-tos, role-based learning paths).

Functional Requirements

- Ingest multiple meeting notes
- Detect recurring themes
- Extract structured steps & definitions
- Render into markdown or LMS-ready format
- Track changes over time

Deliverables

- Script or app generating modules
- Versioning + topic mapping
- Knowledge base organized by roles/projects

Possible Public Datasets

- <https://youtu.be/k8K6wQLxooU?si=GJH1BI44XWobrDZ4>
- <https://www.youtube.com/playlist?list=PLFGfEINsQthZ12EUkq3N9QIThvkf3WGnZ>
- <https://www.kaggle.com/datasets/abhishekunnam/meeting-transcripts>
- <https://github.com/google-research-datasets/MISeD>
- <https://aclanthology.org/2023.bea-1.44.pdf>